

PES-VCS Lab Report

Sidhant Holla SRN: PES1UG24AM466

github.com/Sidhant-holla/PES1UG24AM466-pes-vcs

Phase 1: Object Storage Foundation

Screenshot 1A — ./test_objects

```
1 File changed, 17 insertions(+), 5 deletions(-)
minesid@ValidNewt6:/mnt/c/Users/Sidha/Desktop/PES1UG24AM466-pes-vcs$ make test_objects
./test_objects
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
minesid@ValidNewt6:/mnt/c/Users/Sidha/Desktop/PES1UG24AM466-pes-vcs$
```

Screenshot 1B — Object Store Directory Structure

```
minesid@ValidNewt6:/mnt/c/Users/Sidha/Desktop/PES1UG24AM466-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
minesid@ValidNewt6:/mnt/c/Users/Sidha/Desktop/PES1UG24AM466-pes-vcs$
```

Phase 2: Tree Objects

Screenshot 2A — ./test_tree

```
1 file changed, 3 insertions(+), 2 deletions(-)
minesid@ValidNewt6:/mnt/c/Users/Sidha/Desktop/PES1UG24AM466-pes-vcs$ make test_tree
./test_tree
make: 'test_tree' is up to date.
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
minesid@ValidNewt6:/mnt/c/Users/Sidha/Desktop/PES1UG24AM466-pes-vcs$
```

Screenshot 2B — Raw Tree Object (xxd)

```
minesid@ValidNewt6:/mnt/c/Users/Sidha/Desktop/PES1UG24AM466-pes-vcs$ find .pes/objects -type f
.pes/objects/2c/f8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4
minesid@ValidNewt6:/mnt/c/Users/Sidha/Desktop/PES1UG24AM466-pes-vcs$ xxd .pes/objects/2c/f8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4 | head -20
00000000: 626c 6f62 2036 0068 656c 6c6f 0a      blob 6.hello.
minesid@ValidNewt6:/mnt/c/Users/Sidha/Desktop/PES1UG24AM466-pes-vcs$
```

Phase 3: The Index (Staging Area)

Screenshot 3A — pes init → pes add → pes status

```
minesid@ValidNewt6:/mnt/c/Users/Sidha/Desktop/PES1UG24AM466-pes-vcs$ make pes
./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
Initialized empty PES repository in .pes/
Staged changes:
  staged:    file1.txt
  staged:    file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked: .gitignore
  untracked: commit.c
  untracked: commit.h
  untracked: index.c
  untracked: index.h
  untracked: Makefile
  untracked: object.c
  untracked: pes.c
  untracked: pes.h
  untracked: README.md
  untracked: test_objects
  untracked: test_objects.c
  untracked: test_sequence.sh
  untracked: test_tree
  untracked: test_tree.c
  untracked: tree.c
  untracked: tree.h
```

```
minesid@ValidNewt6:/mnt/c/Users/Sidha/Desktop/PES1UG24AM466-pes-vcs$
```

Screenshot 3B — cat .pes/index

```
minesid@ValidNewt6:/mnt/c/Users/Sidha/Desktop/PES1UG24AM466-pes-vcs$ cat .pes/index
100777 2cf8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4 1776272737 6 file1.txt
100777 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776272737 6 file2.txt
minesid@ValidNewt6:/mnt/c/Users/Sidha/Desktop/PES1UG24AM466-pes-vcs$
```

Phase 4: Commits and History

Screenshot 4A — pes log

```
minesid@ValidNewt6:/mnt/c/Users/Sidha/Desktop/PES1UG24AM466-pes-vcs$ ./pes init
echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"

echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"

echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"

./pes log
Initialized empty PES repository in .pes/
Committed: 6628fdb3eae3... Initial commit
Committed: ff24efa1c735... Add world
Committed: 8dfa994f575e... Add farewell
commit 8dfa994f575e9b1b31a3acc9df6cb2c32dfc076a227915a5406a3cb616d7b648
Author: Sidhant Holla <PES1UG24AM466>
Date: 1776274360

    Add farewell

commit ff24efa1c735a2bbb70193850c807c8d7926f62a2b9d56215ed40c6601f47c26
Author: Sidhant Holla <PES1UG24AM466>
Date: 1776274360

    Add world

commit 6628fdb3eae344a86c3711546b9ba98f5efb99dd5ffc75935d4e506c42441bb2
Author: Sidhant Holla <PES1UG24AM466>
Date: 1776274360

    Initial commit

commit 341d784f3caf6ddc3545506270e17952b39ebaa8ec2410f49475c93033880a6e
Author: PES User <pes@localhost>
Date: 1776273798

    Initial committial commiQ

minesid@ValidNewt6:/mnt/c/Users/Sidha/Desktop/PES1UG24AM466-pes-vcs$
```

Screenshot 4B — find .pes -type f | sort

```
minesid@ValidNewt6:/mnt/c/Users/Sidha/Desktop/PES1UG24AM466-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/34/1d784f3caf6ddc3545506270e17952b39ebaa8ec2410f49475c93033880a6e
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/66/28fdb3eae344a86c3711546b9ba98f5efb99dd5ffc75935d4e506c42441bb2
.pes/objects/8d/fa994f575e9b1b31a3acc9df6cb2c32dfc076a227915a5406a3cb616d7b648
.pes/objects/a4/34f8b1bf184edcfb0a2eb35fd6d405ae02fa246b7205d26ad9f5c087266025
.pes/objects/d6/dfa358f4da1a003e5a16a953e496962f93ecb6809f767d8bbfc3a926a0b58
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/ea/abeb710475acf500f6636b1507e573174a5a47d09906b1917edc5fe4fb17fb
.pes/objects/ff/24efa1c735a2bbb70193850c807c8d7926f62a2b9d56215ed40c6601f47c26
.pes/refs/heads/main
minesid@ValidNewt6:/mnt/c/Users/Sidha/Desktop/PES1UG24AM466-pes-vcs$
```

Screenshot 4C — HEAD and Branch Reference

```

.pes/refs/heads/main
minesid@ValidNewt6:/mnt/c/Users/Sidha/Desktop/PES1UG24AM466-pes-vcs$ cat .pes/refs/heads/main
8dfa994f575e9b1b31a3acc9df6cb2c32dfc076a227915a5406a3cb616d7b648
minesid@ValidNewt6:/mnt/c/Users/Sidha/Desktop/PES1UG24AM466-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
minesid@ValidNewt6:/mnt/c/Users/Sidha/Desktop/PES1UG24AM466-pes-vcs$

```

Figure 4C: .pes/refs/heads/main contains the latest commit hash; .pes/HEAD contains the symbolic ref pointing to that branch.

Final: Integration Test

```

minesid@ValidNewt6:/mnt/c/Users/Sidha/Desktop/PES1UG24AM466-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:    file.txt
  staged:    hello.txt

Unstaged changes:
(nothing to show)

Untracked files:
(nothing to show)

--- First Commit ---
Committed: d3d011adfb9e... Initial commit

Log after first commit:
commit d3d011adfb9e878615a3a9d981bd8d8d33314b1a16748fe2b108131cb0948f9
Author: Sidhant Holla <PES1UG24AM466>
Date:   1776274485

    Initial commit♦♦

--- Second Commit ---
Committed: 6917f0b74a27... Update file.txt

--- Third Commit ---
Committed: c515cfb40cb9... Add farewell

--- Full History ---
commit c515cfb40cb9fbc0439ccb9148c83a88943b0f126fa3410536acab83e8275bcf
Author: Sidhant Holla <PES1UG24AM466>

```


--- Full History ---

commit c515cfb40cb9fbc0439ccb9148c83a88943b0f126fa3410536acab83e8275bcf

Author: Sidhant Holla <PES1UG24AM466>

Date: 1776274485

Add farewell

commit 6917f0b74a27270d9827e5f717ab96bd946af42c4335e30d9361bd66bea0068d

Author: Sidhant Holla <PES1UG24AM466>

Date: 1776274485

Update file.txtter auth!

commit d3d011adfb9e878615a3a9d981bd8d88d33314b1a16748fe2b108131cb0948f9

Author: Sidhant Holla <PES1UG24AM466>

Date: 1776274485

Initial commitQ

--- Reference Chain ---

HEAD:

ref: refs/heads/main

refs/heads/main:

c515cfb40cb9fbc0439ccb9148c83a88943b0f126fa3410536acab83e8275bcf

--- Object Store ---

Objects created:

Objects created:

10

.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d

.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa

.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d

Objects created:

10

.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d

.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa

.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d

Objects created:

10

.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d

.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa

.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d

.pes/objects/69/17f0b74a27270d9827e5f717ab96bd946af42c4335e30d9361bd66bea0068d

pes-vcs  Debug    

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

Objects created:
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/69/17f0b74a27270d9827e5f717ab96bd946af42c4335e30d9361bd66bea0068d
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/c5/15cfb40cb9fbc0439ccb9148c83a88943b0f126fa3410536acab83e8275bcf
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48baf6aa3829f61f75b8d10d5350f9
.pes/objects/d3/d011adfb9e878615a3a9d981bd8d88d33314b1a16748fe2b108131cb0948f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
minesid@ValidNewt6:/mnt/c/Users/Sidha/Desktop/PES1UG24AM466-pes-vcs$
```

Phase 5 & 6: Analysis Questions

Q5.1 — How would you implement pes checkout <branch>?

A branch is literally just a file in `.pes/refs/heads/` with a commit hash in it, so creating a branch is just creating a file. For checkout the steps are:

In `.pes/`:

- Rewrite HEAD to say "ref: refs/heads/<branch>" so it now points to the new branch.
- If the branch doesn't exist yet, create `.pes/refs/heads/<branch>` with the current commit hash.

In the working directory:

- Read the target branch's commit to get its tree hash. Walk that tree recursively.
- For each file in the target tree that differs from what's currently on disk, overwrite it with the blob contents from the object store.
- Delete any files that exist in the current tree but not the target.
- Create any files that exist in the target but not on disk currently.

What makes it hard: the main complexity is handling dirty files. If the user has modified a file that also differs between branches, you can't just overwrite it — that would silently throw away their work. So you need to check each file against both the index and the target tree before touching anything. Another issue is atomicity — if something fails halfway through, the working directory ends up in an inconsistent state, so ideally you'd do a dry-run validation pass first.

Q5.2 — Detecting dirty working directory conflicts

The goal is to figure out: does this file have local changes, AND would switching branches overwrite it with something different?

- First, compare the working file's mtime and size to what's stored in the index. If they match, the file almost certainly hasn't changed since it was staged — no need to hash it.
- If mtime or size differs, actually hash the file and compare to the blob hash in the index. If they're different, the file is dirty.
- Then check whether the target branch even cares about this file: read the target commit, walk its tree, and look up the blob hash for this path. If it matches the index hash, the branch switch won't change this file anyway — no conflict.
- A conflict only exists if both things are true: the file is dirty locally, AND the target branch has a different version of it. If the file is dirty but both branches have the same version, it's fine.

Q5.3 — Detached HEAD

Detached HEAD just means HEAD contains a raw commit hash instead of something like "ref: refs/heads/main". You end up here when you checkout a specific commit directly.

If you commit in this state, the commits do get written to the object store correctly. The difference is that `head_update` writes the new hash directly into HEAD rather than updating a branch file. So the commits exist but nothing named points to them — they won't show up in `pes log` from any branch, and eventually GC will delete them.

Recovery: if you still have the hash from your terminal output, you can manually create a branch pointing to it by writing that hash into `.pes/refs/heads/some-branch-name`. Real git makes this easier through the `reflog` which keeps a history of everywhere HEAD has pointed, but since PES-VCS doesn't have that, you're relying on your terminal history.

Q6.1 — Garbage Collection Algorithm

The basic algorithm is mark-and-sweep:

- Start from all branch tips (every file in `.pes/refs/heads/`) as the root set.
- Do a graph traversal from each root — for every commit you visit, follow its parent and tree pointers. For every tree, follow its subtree and blob entries. Add every hash you visit to a "reachable" set.

- Walk the entire `.pes/objects/` directory. For every object file, reconstruct its hash from the path. If it's not in the reachable set, delete it.

For the reachable set, a hash set gives $O(1)$ lookups which is what you want when you're checking millions of objects. A sorted array of 32-byte hashes with binary search also works well since the comparison is just a fixed-size memcmp.

For 100k commits and 50 branches: you'd visit all 100k commits following parent chains, plus roughly 60 objects per commit (1 root tree, ~10 subtrees, ~50 blobs, though deduplication means many are shared). That's maybe 5-6 million object visits but far fewer unique objects in the store — probably under 2 million in practice.

Q6.2 — GC Race Condition

The dangerous scenario:

- A commit operation writes a new blob to the store — call it abc123 — but hasn't updated the branch ref yet.
- GC runs at this exact moment, collects all branch tips, and walks reachable objects. Since the ref update hasn't happened, abc123 isn't reachable from anything.
- GC deletes abc123.
- The commit continues, builds a tree pointing to abc123, and updates the branch ref.
- Now the repo has a commit that references a missing object. Reading it will fail.

How Git deals with it: the main protection is a grace period — `git prune` won't delete any object newer than 2 weeks by default. Since commits finish in milliseconds, anything recently written is safe regardless of whether GC catches it mid-operation. Git also uses lock files to prevent concurrent ref updates, and it packs reachable objects into pack files before pruning loose ones, which shrinks the window for races even further.